

IT7075: Applied AI for Cybersecurity

Instructor: Chengcheng Li • University of Cincinnati

School of Information Technology

Tools & Environment

Project — AI Tools and Environment Setup

A competency-based assessment of the development environment used throughout this course.

Local machine, Google Colab, and Jetstream2 • self-directed • approximately 4–6 hours • 100 points

1. Overview

This project establishes the technical foundation for the remainder of the course. Students configure the environments in which all subsequent modules are completed: a local editor and isolated Python environment, a version-controlled repository, a hosted notebook platform with persistent storage, programmatic access to a hosted language model, a cloud virtual machine reached over SSH, and an isolated virtualization laboratory.

Assessment is competency-based: credit is awarded for demonstrated mastery of each skill, evidenced in a written report and a recorded demonstration. Little programming is involved; the difficulty lies in installation, configuration, verification, and explanation. No step-by-step installation instructions are provided — students locate and follow current product documentation themselves, and generative AI assistants may be used freely to install, configure, and troubleshoot. Assessment nevertheless rests on the student's own understanding: each student must be able to explain the configuration decisions made and to operate the completed environment unaided during the recorded demonstration.

2. Learning objectives

On completion of this project, students will be able to:

1. Configure a local development environment in which code executes inside an isolated Python environment.
2. Manage a project under version control and exclude credentials from the repository.
3. Work in a hosted notebook environment with persistent storage, and obtain programmatic access to a hosted language model using credentials loaded from configuration rather than embedded in source code.
4. Provision and use remote and virtualized machines, authenticating by SSH key and controlling network reachability between host and guest.
5. Inventory the computational resources available on each platform, and document and present a technical configuration process in written and recorded form.

Prerequisites. None. Students require administrative rights on a personal computer, a Google account, and a GitHub account.

3. Required competencies

Each competency states the capabilities to be demonstrated and the evidence required. Evidence must show the command or interface action that produced each result. Capture evidence as the work proceeds.

3.1 Integrated development environment (8 points). Install Visual Studio Code, open a local project directory, install and verify the Python, Jupyter, and GitHub extensions, and execute both a script and a notebook cell within the editor. *Evidence:* the extension list, the open project directory, and script and notebook output.

3.2 Isolated Python environments (10 points). Create, activate, and deactivate a virtual environment; install a dependency set from a requirements file (`requirements.txt`); and select the environment as interpreter and as notebook kernel. *Evidence:* the commands issued; the output of `python -V`, which `python` (or `where python`), and `pip list`; and the interpreter selection, whose path must resolve inside the environment directory rather than the system installation.

3.3 Version control and remote repositories (10 points). Configure the local Git identity; create and clone a personal public course repository; commit and push from the editor; and exclude `.env` and `.venv/` by means of `.gitignore`. *Evidence:* the repository URL, at least two substantive commits, the source-control panel showing the remote, `.gitignore`, and `git status` output that does not list `.env`.

3.4 Hosted notebooks and persistent storage (10 points). Execute a notebook in Google Colab, mount Google Drive, write a file to Drive, and confirm that it survives a runtime restart. *Evidence:* executed cells with output, the mount confirmation, and the file read back after the runtime is reconnected.

3.5 Programmatic model access with managed credentials (12 points). Obtain an API key from an approved provider such as OpenAI and Anthropic, configure it locally in a `.env` file and in Colab under Secrets. *Evidence:* the model response from both environments and the credential-loading code. No key may appear in any submitted material.

3.6 Cloud virtual machine deployment on Jetstream2 (10 points). Create a virtual machine on Jetstream2; generate an SSH public and private key pair; add the public key to the remote system and configure passwordless authentication; store the private key securely on the local machine; create an SSH host entry in `~/.ssh/config` for simplified access; connect through VS Code Remote-SSH and use the instance as a development environment; and shelve the instance when it is not in use. *Evidence:* the running instance, an SSH login completed without a password prompt, the SSH host configuration, and VS Code showing the remote connection in its status bar.

Contingency

This competency depends on course computing resources and grant approval. If Jetstream2 is not available, these exercises are not required and are not graded, and the ten points are redistributed proportionally across the remaining competencies. Availability will be announced.

3.7 Virtualization and isolated targets (12 points). Install VirtualBox, VMware, or Docker; deploy Metasploitable2, or a containerized target such as DVWA or OWASP Juice Shop where 32-bit x86 guests are unsupported; configure host-only or NAT networking; and verify host-to-guest reachability. *Evidence:* the running guest, the network configuration, a successful `ping` together with one service reached from the host, and a justification of the network mode selected.

3.8 Computational resource inventory (8 points). Determine processor, memory, and GPU availability on the local machine, in Colab, and on the Jetstream2 instance where available, using both a system utility

(`nvidia-smi` , `system_profiler` , `wmic` , `lspci`) and a framework query (`torch.cuda.is_available()` , or `torch.backends.mps.is_available()` on Apple silicon). *Evidence*: output from each platform. An accurately reported and correctly interpreted absence of a GPU receives full credit.

3.9 Recorded technical demonstration (12 points). Record and narrate for less than 10 minutes, showing the systems of §3.1–3.8 in live operation, and publish. The narration must explain the principal configuration steps, the most significant problem encountered and its resolution, and one decision that would be made differently.

4. Deliverable

Submit this

a single PDF report containing all written material and all screenshots, with the video link on its first page. No other files are submitted.

The remaining **8 points** are awarded for the organization and clarity of the report: evidence labeled by competency number, legible screenshots showing the commands that produced them, and a specific account of problems and resolutions. Credentials must be cropped or redacted from every figure before submission.